

Technische Universität Wien

Fakultät für Elektrotechnik und Informationstechnik

Konzeption und Implementierung eines Python-basierten Planungstools zur Unterstützung der Lehrveranstaltungsplanung

Bachelorarbeit

Verfasser: Nicolas Carraro

Matrikelnummer: 12202357

Betreuer: Univ.Prof. Dipl.-Ing. Dr.techn. Michael Walzl

Wien, 27. Juni 2026

Kurzfassung

Diese Bachelorarbeit beschreibt die Konzeption und Implementierung eines Python-basierten Desktop-Planungstools zur Unterstützung der Lehrveranstaltungsplanung. Ausgangspunkt ist ein manueller Planungsprozess, bei dem Termine, Räume, Lehrpersonen, Gruppen und freie Zeiträume in mehreren Tabellen verwaltet werden. Ziel der Arbeit ist es, diese Informationen zentral zu strukturieren, Termine übersichtlich in Kalenderansichten darzustellen und typische Planungskonflikte besser sichtbar zu machen.

Das entwickelte Tool verwendet PySide6 für die grafische Benutzeroberfläche und speichert Projektdaten lokal in JSON-Dateien. Es unterstützt Tages-, Wochen- und Monatsansichten, Drag-and-Drop zur Terminplanung, Filter- und Suchfunktionen, Serientermine, Import- und Exportfunktionen sowie eine regelbasierte Konflikterkennung. Die Evaluation erfolgt durch manuelle Tests typischer Arbeitsabläufe. Die Ergebnisse zeigen, dass das Tool den Planungsprozess nicht automatisiert ersetzt, aber die Bearbeitung, Kontrolle und Nachvollziehbarkeit der Lehrveranstaltungsplanung verbessert.

Abstract

This bachelor thesis describes the design and implementation of a Python-based desktop planning tool for supporting course scheduling. The starting point is a manual planning process in which appointments, rooms, lecturers, groups and free periods are managed across several tables. The goal of the thesis is to structure this information centrally, present appointments in calendar views and make typical scheduling conflicts easier to detect.

The developed tool uses PySide6 for the graphical user interface and stores project data locally in JSON files. It supports day, week and month views, drag-and-drop scheduling, filtering and search, recurring appointments, import and export functions, and rule-based conflict detection. The evaluation is based on manual tests of typical workflows. The results show that the tool does not replace the planning decisions themselves, but improves editing, checking and traceability in course scheduling.

Inhaltsverzeichnis

Kurzfassung	1
Abstract	1
Abkürzungsverzeichnis	6
1 Einleitung	7
1.1 Problemstellung	7
1.2 Zielsetzung	7
1.3 Forschungsfrage	8
1.4 Aufbau der Arbeit	9
2 Grundlagen und Kontext	10
2.1 Lehrveranstaltungsplanung an einem Institut	10
2.2 Herausforderungen manueller Planungsprozesse	10
2.3 Softwaregestützte Planung	11
2.4 Eingesetzte Technologien	11
2.4.1 Python	12
2.4.2 PySide6 und Qt	12
2.4.3 JSON als Datenformat	13
3 Anforderungsanalyse	14
3.1 Ableitung der Anforderungen	14
3.2 Funktionale Anforderungen	14
3.3 Nicht-funktionale Anforderungen	14
3.4 Abgrenzung der Software	14
4 Konzeption des Planungstools	16
4.1 Gesamtarchitektur der Anwendung	16
4.2 Aufbau der Benutzeroberfläche	17
4.3 Datenmodell und Datenbeziehungen	18
4.4 Planungsworkflow	18
4.5 Aktualisierung und Zusammenspiel der Komponenten	19
5 Implementierung	20
5.1 Entwicklungsumgebung und Projektstruktur	20
5.2 Hauptfenster und zentrale Steuerung	20
5.3 Kalenderansichten	22
5.4 Termin- und Stammdatenverwaltung	24
5.5 Drag-and-Drop-Funktionalität	25

5.6	Filterfunktion	28
5.7	Konflikterkennung	29
5.8	Import, Export und Speicherung	31
5.9	Semester-Werkzeuge	33
5.10	Undo/Redo, Shortcuts und Layout-Verwaltung	33
6	Test und Evaluation	35
6.1	Teststrategie	35
6.2	Durchführung der Tests	35
6.3	Bewertung der Ergebnisse	40
7	Diskussion	41
7.1	Zielerreichung	41
7.2	Grenzen der Software	41
7.3	Erweiterungsmöglichkeiten	41
8	Fazit und Ausblick	42

Abbildungsverzeichnis

1	Hauptfenster des Planungstools mit Kalenderbereich und Dock-Bereichen .	17
2	Tagesansicht mit zeitlicher Einordnung der Termine	22
3	Dateneditor zur Verwaltung von Terminen und Stammdaten	24
4	Terminliste mit noch nicht vollständig eingeplanten Terminen	26
5	Drag-and-Drop-Vorschau eines Termins mit erkanntem Konflikt	27
6	Konfliktanzeige mit aufgelisteten Planungskonflikten	30

Tabellenverzeichnis

1	Funktionale Anforderungen an das Planungstool	15
2	Nicht-funktionale Anforderungen an das Planungstool	16
3	Manuelle Tests der Termin- und Stammdatenverwaltung	35
4	Manuelle Tests der Kalenderansichten und Drag-and-Drop-Funktion	36
5	Manuelle Tests der Filterung und Aktualisierung	36
6	Manuelle Tests der Konflikterkennung	37
7	Manuelle Fehler- und Validierungstests	38
8	Manuelle Tests zu Speicherung, Import/Export und Bedienhilfen	39

Abkürzungsverzeichnis

CSV	Comma-Separated Values
CRUD	Create, Read, Update, Delete
ETIT	Elektrotechnik und Informationstechnik
JSON	JavaScript Object Notation
LVA	Lehrveranstaltung
SS	Sommersemester
WS	Wintersemester
TISS	TU Wien Informations-Systeme und Services
UI	User Interface

1 Einleitung

1.1 Problemstellung

Es werden jährlich neue Lehrveranstaltungen, Übungen, Labore und Klausuren eingeplant. Dazu müssen bei den Entscheidungen, wann welcher Termin eingeplant wird, mehrere verschiedene Informationen gleichzeitig berücksichtigt werden:

- zeitliche Überschneidungen von Lehrveranstaltungen, Lehrpersonen, Räumen und Gruppen
- Raumkapazitäten
- freie Tage und Feiertage

Diese Planung wird durch die hohe Menge an individuellen Terminen eine komplexe Aufgabe. Die manuelle Planung erfolgt bisher über Excel-Tabellen, welche die Verschiebung oder Veränderung einzelner Termine nicht effizient erlauben. Durch die manuelle Planung können auch weitere Probleme auftreten wie doppelte oder uneinheitliche Daten, Änderungen müssen an mehreren Stellen angepasst werden, schwer erkennbare Konflikte und ein generell schlechter Überblick über die einzelnen Termine.

Beispiele für mögliche Konflikte könnten sein: doppelt belegte Räume, Lehrperson zeitgleich eingeplant oder ein Termin an einem Feiertag angesetzt.

Aufgrund dieser Probleme und dadurch dass dies eine jährlich wiederkehrende Aufgabe ist besteht ein Bedarf an einer zentralen Softwarelösung. Diese Software soll die Planung der Termine übersichtlicher, strukturierter und nachvollziehbarer machen. Daher geht es in dieser Arbeit um die Entwicklung eines Python-basierten Planungstools für die Unterstützung der LVA-Jahresplanung. Der Fokus liegt dabei auf der praktischen Anwendbarkeit dieser Software am Institut.

1.2 Zielsetzung

Ziel dieser Bachelorarbeit ist die Konzeption, Implementierung und Bewertung eines Planungstools. Dieses Tool soll den jährlichen Planungsprozess der LVA-Termine digital unterstützen und damit effizienter und einfacher gestalten. Zentrale Ziele dieser Software sind:

- strukturierte und zentrale Verwaltung von Planungsdaten
- übersichtliche Darstellung von Terminen
- einfaches Verschieben und Zuweisen von Terminen mittels Drag-and-Drop
- bessere Erkennung von Planungskonflikten

- Filterung nach relevanten Kriterien
- Import und Export von Planungsdaten

Die Umsetzung soll in Form einer mit Python entwickelten Desktop-Anwendung erfolgen. Die Entwicklung einer einfachen, intuitiven grafischen Benutzeroberfläche ist dabei besonders wichtig, da die Bedienung dieses Tools auch ohne Programmierkenntnisse möglich sein soll. Benutzerfreundlichkeit wird in dieser Arbeit als Eigenschaft verstanden, die beschreibt, ob bestimmte Nutzer ihre Ziele in einem bestimmten Nutzungskontext effektiv, effizient und zufriedenstellend erreichen können [8].

Ziel dieser Anwendung ist nicht die gesamte Planung zu übernehmen, sondern es soll als Unterstützung für den Nutzer dienen, der die Planung durchführt. Es soll besonders auf Benutzerfreundlichkeit, Nachvollziehbarkeit und Erweiterbarkeit geachtet werden.

1.3 Forschungsfrage

Aus der beschriebenen Problemstellung und Zielsetzung der Arbeit und dem Bedarf nach einer digitalen Unterstützung für die LVA-Jahresplanung ergibt sich die Frage, wie man ein Werkzeug entwickelt, das Planungsdaten strukturiert verwaltet, Termine übersichtlich darstellt und mögliche Konflikte leichter erkennbar macht.

Die zentrale Forschungsfrage dieser Arbeit lautet daher:

Wie kann ein intuitives Python-basiertes Desktop-Planungstool entwickelt werden, um die jährliche Lehrveranstaltungsplanung übersichtlicher, strukturierter, effizienter und weniger fehleranfällig zu gestalten?

Um diese Frage zu beantworten werden auch zusätzliche Teilfragen betrachtet:

- Welche Anforderungen ergeben sich aus dem Planungsprozess?
- Wie können Termine, Lehrveranstaltungen, Räume und weitere Daten sinnvoll strukturiert und verwaltet werden?
- Wie kann eine Benutzeroberfläche gestalten, sodass Termine übersichtlich dargestellt, gefiltert und mittels Drag-and-Drop verschoben werden können?
- Wie können typische Planungskonflikte erkannt und für den Nutzer schnell sichtbar gemacht werden?
- Inwiefern erfüllt das entwickelte Tool die definierten Anforderungen?

1.4 Aufbau der Arbeit

Die Arbeit beginnt mit einer Beschreibung der Probleme, die bei der jährlichen Planung von Lehrveranstaltungen entstehen. Darauf werden die Ziele der Arbeit und die Forschungsfrage vorgestellt.

Anschließend werden die Grundlagen und Anforderungen beschrieben. Danach folgen Konzeption und Implementierung des Tools. Zum Schluss werden Tests, Ergebnisse, Grenzen und mögliche Erweiterungen dargestellt.

2 Grundlagen und Kontext

2.1 Lehrveranstaltungsplanung an einem Institut

Bei der Lehrveranstaltungsplanung geht es darum, die verschiedenen Lehrveranstaltungen eines Instituts in einen passenden zeitlichen und organisatorischen Rahmen zu bringen. Dazu müssen Veranstaltungen wie Vorlesungen, Übungen, Labore oder Prüfungen konkreten Terminen, Räumen und teilweise auch bestimmten Gruppen zugeordnet werden. Die Planung ist daher mehr als nur das Eintragen von Terminen in einen Kalender.

Je nach Art der Lehrveranstaltung können dabei unterschiedliche Anforderungen entstehen. Eine Vorlesung benötigt zum Beispiel oft einen größeren Raum, während ein Labor an bestimmte Ausstattung gebunden sein kann. Prüfungen wiederum finden häufig nur an einzelnen Terminen statt und müssen besonders klar geplant werden. Auch die Dauer der Termine und die Anzahl der teilnehmenden Personen können sich je nach Veranstaltung unterscheiden. Solche Anforderungen entsprechen typischen Einschränkungen der Lehrveranstaltungsplanung, bei der Veranstaltungen bestimmten Zeitfenstern und Räumen zugeordnet werden müssen [1].

Ein Termin steht dabei meistens nicht für sich allein, sondern ist mit mehreren anderen Informationen verbunden. Dazu gehören unter anderem die Lehrveranstaltung selbst, das Semester, die zuständige Lehrperson und der Raum. Dadurch entsteht eine Struktur, bei der viele Daten miteinander zusammenhängen. Für ein Institut ist es wichtig, dass aus diesen Informationen ein übersichtlicher und nachvollziehbarer Plan entsteht.

In dieser Arbeit wird die Lehrveranstaltungsplanung deshalb als ein Prozess verstanden, bei dem verschiedene Planungsdaten gesammelt, miteinander verknüpft und für die weitere Bearbeitung nutzbar gemacht werden. Diese Sichtweise bildet die Grundlage für die spätere Entwicklung des Planungstools.

2.2 Herausforderungen manueller Planungsprozesse

Bei der bisherigen Planung werden viele Informationen in Tabellen gesammelt. Das ist am Anfang praktisch, weil Tabellen schnell erstellt und angepasst werden können. Wenn jedoch viele Termine eingeplant werden müssen, wird es mit der Zeit schwer, den Überblick zu behalten. Besonders bei der Jahresplanung kommen viele einzelne Termine zusammen, die sich gegenseitig beeinflussen können.

Ein Problem ist, dass Änderungen nicht immer nur eine Stelle betreffen. Wenn zum Beispiel ein Termin verschoben wird, muss geprüft werden, ob der neue Zeitpunkt mit anderen Terminen zusammenpasst. Außerdem muss kontrolliert werden, ob der Raum frei ist und ob die betroffene Lehrperson zu diesem Zeitpunkt nicht bereits anders eingeplant ist. Diese Kontrolle muss bei einer manuellen Planung selbst gemacht werden.

Dadurch wird die manuelle Planung mit zunehmender Anzahl an Terminen immer

aufwendiger. Tabellen bleiben zwar flexibel, bieten aber nur begrenzte Unterstützung dabei, Termine übersichtlich darzustellen und mögliche Konflikte frühzeitig zu erkennen.

2.3 Softwaregestützte Planung

Eine softwaregestützte Planung unterstützt den Planungsprozess vor allem dadurch, dass viele einzelne Informationen besser zusammenzuführen werden. Anstatt Termine nur in Tabellen einzutragen, können sie in einer Anwendung direkt mit weiteren Daten verbunden werden, zum Beispiel mit einer Lehrveranstaltung oder einem Raum.

Ein Vorteil ist, dass die Daten an einer zentralen Stelle verwaltet werden können. Wenn ein Termin geändert wird, muss diese Änderung nicht in mehreren Tabellen oder Dateien nachgetragen werden. Die angepassten Daten können direkt in der Anwendung angezeigt und weiterverwendet werden.

Auch die Darstellung spielt eine wichtige Rolle. Eine Kalenderansicht ist für die Planung oft verständlicher als eine reine Tabelle, weil Termine zeitlich eingeordnet werden können. So sieht man schneller, an welchen Tagen bereits viele Termine liegen oder wo noch freie Zeiträume vorhanden sind. Durch Filter kann zusätzlich gezielt nach bestimmten Lehrveranstaltungen, Räumen, Lehrpersonen, etc. gesucht werden.

Besonders hilfreich ist außerdem eine direkte Bearbeitung der Termine über die Benutzeroberfläche. Wenn Termine zum Beispiel per Drag-and-Drop verschoben oder zugewiesen werden, wird die Planung übersichtlich und schnell angepasst. Statt Werte manuell in Tabellenzellen zu ändern, kann der Nutzer einen Termin direkt an die passende Stelle ziehen. Dadurch wird der Planungsprozess näher an der tatsächlichen Kalenderansicht durchgeführt.

Eine Software kann außerdem dabei helfen, mögliche Konflikte sichtbarer zu machen. Wenn ein Raum bereits belegt ist oder eine Lehrperson zur gleichen Zeit in einem anderen Termin vorkommt, kann das Programm solche Fälle erkennen und anzeigen. Die Entscheidung, wie mit einem Konflikt umgegangen wird, bleibt weiterhin beim Nutzer. Die Software übernimmt also nicht die gesamte Planung, sondern unterstützt dabei, Fehler schneller zu finden und Änderungen übersichtlicher umzusetzen.

2.4 Eingesetzte Technologien

Für die Umsetzung des Planungstools wurden mehrere Technologien verwendet. Die Anwendung wurde mit Python entwickelt und als Desktop-Programm umgesetzt. Für die grafische Oberfläche kommt PySide6 zum Einsatz, da damit die verschiedenen Fenster, Tabellen und Bedienbereiche der Anwendung erstellt werden können.

Die Daten des Planungstools werden in JSON-Dateien gespeichert. Dadurch können die einzelnen Planungsdaten strukturiert abgelegt und beim Start der Anwendung wieder geladen werden. Für den Datenaustausch mit externen Beteiligten wird zusätzlich ein

Excel-Format unterstützt, damit bestehende Excel-basierte Arbeitsabläufe weiterverwendet werden können.

Die wichtigsten verwendeten Technologien werden in den folgenden Abschnitten kurz beschrieben.

2.4.1 Python

Das Planungstool wurde mit Python entwickelt [3]. Python bildet dabei die Grundlage für die eigentliche Logik der Anwendung, also zum Beispiel für das Laden und Speichern der Daten, das Bearbeiten von Terminen und das Prüfen von möglichen Konflikten.

Für dieses Projekt war Python passend, weil die Sprache gut lesbar ist und sich der Code übersichtlich strukturieren lässt. Ein weiterer Grund für die Verwendung von Python ist, dass die Sprache weit verbreitet ist und von vielen Personen verwendet wird. Dadurch kann die spätere Wartung oder Erweiterung des Tools erleichtert werden, da sich neue Entwicklerinnen und Entwickler schneller in den Code einarbeiten können.

Außerdem unterstützt Python viele Bibliotheken. Für dieses Projekt war vor allem PySide6 wichtig, weil damit die grafische Oberfläche erstellt wurde. Dadurch konnten die verschiedenen Fenster, Tabellen und Bedienelemente des Planungstools umgesetzt werden.

2.4.2 PySide6 und Qt

Für die grafische Benutzeroberfläche des Planungstools wurde PySide6 verwendet. PySide6 verbindet Python mit dem Qt-Framework. Qt bietet viele Elemente, die für Desktop-Anwendungen gebraucht werden, zum Beispiel Fenster, Tabellen, Buttons, Menüs und Dialoge [4]. Dadurch konnte das Tool als lokale Anwendung mit einer eigenen Benutzeroberfläche umgesetzt werden.

Grundsätzlich hätte die Oberfläche auch mit PyQt entwickelt werden können, da PyQt und PySide6 ähnliche Funktionen bereitstellen und beide auf Qt basieren. Für dieses Projekt wurde jedoch PySide6 gewählt, da es das offizielle Qt-for-Python-Binding ist [5]. Außerdem kann die LGPL-Lizenz von PySide6 bei eigenständigen Anwendungen flexibler sein.¹

Im Planungstool wird PySide6 vor allem für die verschiedenen Bedienbereiche der Anwendung genutzt. Dazu gehören zum Beispiel Kalenderansichten, Tabellen, Eingabefenster, Filterbereiche und Listen.

Wenn der Nutzer einen Termin auswählt, einen Filter setzt oder Daten in einem Dialog bearbeitet, muss die Anwendung auf diese Eingaben reagieren. PySide6 stellt dafür die Verbindung zwischen der Benutzeroberfläche und der Programmlogik her. Dadurch können Änderungen direkt verarbeitet und an anderen Stellen der Anwendung sichtbar gemacht werden.

¹<https://www.qt.io/development/open-source-lgpl-obligations>

2.4.3 JSON als Datenformat

Die Planungsdaten werden in mehreren JSON-Dateien eines Projektordners gespeichert. Dazu gehören Termine, Lehrveranstaltungen, Räume, Studienrichtungen und freie Zeiträume. Die regulären Sommer- und Wintersemester werden dagegen aus Semester-ID und Jahr erzeugt. Die Studiensemester werden global in `src/studiensemester.json` verwaltet. JSON wurde gewählt, weil sich damit strukturierte Daten unkompliziert speichern, wieder laden und bei Bedarf auch direkt in der Datei manuell bearbeiten lassen [2].

Da die JSON-Dateien auch außerhalb des Programms geöffnet werden können, bleibt nachvollziehbar, welche Daten gespeichert wurden. Das kann zum Beispiel hilfreich sein, wenn Fehler gesucht werden oder überprüft werden soll, ob Daten richtig übernommen wurden.

Grundsätzlich hätten für die Speicherung auch CSV-Dateien verwendet werden können. CSV eignet sich vor allem für einfache tabellarische Daten. Für das Planungstool ist das jedoch nicht immer passend, da die Daten teilweise stärker miteinander verknüpft sind. Ein Termin kann zum Beispiel mehrere Eigenschaften oder Verweise auf andere Daten enthalten, wie auf eine LVA oder einen Raum. JSON ist dafür besser geeignet, weil zusammenhängende Informationen als Objekte gespeichert werden können. Dadurch lassen sich die Planungsdaten strukturierter abbilden als in einer einfachen Tabelle.

Für die aktuelle Version und Größe der Anwendung ist diese Art der Speicherung ausreichend. Es ist also kein zentraler Server oder eine Datenbank notwendig. Für den Datenaustausch unterstützt das Tool Excel- und JSON-Projektdateien. Zusätzlich können Raum- und LVA-Listen aus Excel- oder CSV-Dateien übernommen werden. Für Lehrende können gefilterte Terminlisten oder Wochenkalender als Excel-Datei exportiert werden. Wenn das Tool später erweitert wird, könnte eine Datenbank jedoch eine sinnvolle Weiterentwicklung sein.

3 Anforderungsanalyse

3.1 Ableitung der Anforderungen

Die Anforderungen an das Planungstool ergeben sich aus den typischen Arbeitsschritten der jährlichen Lehrveranstaltungsplanung. Dabei müssen Termine nicht nur gespeichert, sondern auch mit Informationen wie Lehrveranstaltungen, Räumen, Semestern oder Gruppen verbunden werden.

Für die Anforderungsanalyse wurden daher vor allem das Anlegen und Bearbeiten von Terminen, das Verschieben von Terminen, das Filtern der Daten und das Erkennen möglicher Konflikte betrachtet. Daraus ergeben sich die folgenden funktionalen und nicht-funktionalen Anforderungen, wie sie in der Softwareentwicklung grundsätzlich unterschieden werden [7].

3.2 Funktionale Anforderungen

Funktionale Anforderungen beschreiben, welche Aufgaben oder Dienste die Software erfüllen bzw. bereitstellen soll [7]. Die wichtigsten funktionalen Anforderungen des Planungstools sind in Tabelle 1 dargestellt.

Die funktionalen Anforderungen bilden damit die wichtigsten Arbeitsschritte der späteren Anwendung ab: Daten erfassen und speichern, Termine planen, Ansichten filtern und Konflikte erkennen.

3.3 Nicht-funktionale Anforderungen

Nicht-funktionale Anforderungen beschreiben Eigenschaften eines Systems, die sich nicht direkt auf einzelne Funktionen beziehen, sondern zum Beispiel Bedienbarkeit, Zuverlässigkeit, Wartbarkeit oder technische Einschränkungen betreffen [7].

Die nicht-funktionalen Anforderungen sind besonders wichtig, weil das Tool nicht nur technisch funktionieren, sondern verständlich und zuverlässig nutzbar sein soll.

3.4 Abgrenzung der Software

Das Planungstool wurde als Unterstützung für die jährliche Lehrveranstaltungsplanung entwickelt. Es soll den Planungsprozess übersichtlicher machen und bei der Bearbeitung der Termine helfen. Die organisatorischen Entscheidungen werden dabei aber alle weiterhin vom Nutzer getroffen.

Nicht Teil dieser Version ist eine automatische Optimierung der gesamten Planung. Die Software erstellt also nicht selbstständig den bestmöglichen Stundenplan, sondern unterstützt den bestehenden Planungsprozess. Auch eine direkte Anbindung an offizielle Universitätssysteme oder eine automatische Raumreservierung ist aktuell nicht umgesetzt.

Anforderung	Beschreibung
Terminverwaltung	Termine sollen erstellt, bearbeitet und gelöscht werden können.
Datenverwaltung	Lehrveranstaltungen, Räume, Studienrichtungen und freie Zeiträume sollen verwaltet werden können. Reguläre Semester werden automatisch erzeugt und Studiensemester als globale Auswahlwerte verwendet.
Terminzuordnung	Termine sollen mit Lehrveranstaltungen, Räumen, Semestern oder Gruppen verbunden werden können.
Kalenderdarstellung	Termine sollen in einer übersichtlichen Kalenderansicht dargestellt werden.
Drag-and-Drop	Termine sollen direkt über die Benutzeroberfläche verschoben oder zugewiesen werden können.
Filterung	Termine sollen nach bestimmten Kriterien gefiltert werden können.
Konflikterkennung	Mögliche Konflikte, wie doppelt belegte Räume oder zeitliche Überschneidungen, sollen erkannt und angezeigt werden.
Speicherung	Planungsdaten sollen gespeichert und beim erneuten Start wieder geladen werden können.
Import und Export	Projektstände sollen als JSON oder Excel ausgetauscht werden können. Zusätzlich sollen Raum- und LVA-Listen aus Excel oder CSV importiert und Terminlisten für Lehrende exportiert werden können.

Tabelle 1: Funktionale Anforderungen an das Planungstool

Die Anwendung ist außerdem als lokale Desktop-Anwendung ausgelegt. Ein gleichzeitiger Zugriff durch mehrere Nutzer ist in dieser Version nicht vorgesehen. Funktionen wie eine direkte Live-Anbindung an TISS oder andere Universitätssysteme, automatische Raumreservierung, automatische Optimierung oder Mehrnutzerbetrieb sind nicht Teil dieser Version und bleiben mögliche Erweiterungen.

Anforderung	Beschreibung
Benutzerfreundlichkeit	Das Tool soll auch ohne Programmierkenntnisse einfach bedienbar sein.
Übersichtlichkeit	Termine und Planungsdaten sollen klar dargestellt werden.
Wartbarkeit	Der Code soll übersichtlich aufgebaut sein, damit Fehler behoben und bestehende Funktionen angepasst werden können.
Erweiterbarkeit	Neue Funktionen sollen später ergänzt werden können.
Nachvollziehbarkeit	Daten und Änderungen sollen verständlich und kontrollierbar bleiben.
Stabilität	Die Anwendung soll Daten zuverlässig speichern und laden können.
Responsivität	Die Anwendung soll auch bei größeren Terminkmengen flüssig reagieren. Rechenintensive Komfortfunktionen sollen deshalb bei Bedarf deaktiviert werden können.
Lokale Nutzbarkeit	Das Tool soll direkt auf einem Computer ausgeführt werden können und keinen Server benötigen.

Tabelle 2: Nicht-funktionale Anforderungen an das Planungstool

4 Konzeption des Planungstools

4.1 Gesamtarchitektur der Anwendung

Das Planungstool wurde als lokale Desktop-Anwendung konzipiert und besteht nicht nur aus einer einzelnen Kalenderansicht, sondern aus mehreren Bereichen, die zusammenarbeiten. Im Mittelpunkt steht das Hauptfenster, das die verschiedenen Teile der Benutzeroberfläche verbindet und die zentralen Abläufe der Anwendung steuert.

Grundsätzlich lässt sich die Anwendung in drei Bereiche einteilen: die Benutzeroberfläche, die Datenverwaltung und die Planungslogik. Die Benutzeroberfläche stellt Kalenderansichten, Listen, Eingabefenster und weitere Bedienelemente bereit. Die Datenverwaltung ist dafür zuständig, Planungsdaten zu laden, zu speichern und für die anderen Teile der Anwendung verfügbar zu machen. Die Planungslogik umfasst Funktionen wie Filterung, Konflikterkennung und die Aktualisierung der Anzeige nach Änderungen.

Der Start der Anwendung erfolgt über die zentrale GUI-Initialisierung. Dabei werden zunächst Einstellungen und Datenpfad geprüft. Anschließend wird das Hauptfenster erstellt, das die zentrale Steuerungsrolle übernimmt. Das Hauptfenster erzeugt den Kalender, die Terminliste, den Dateneditor, den globalen Filterbereich, die Datumsnavigation und die Konflikthanzeige. Damit ist das Hauptfenster nicht nur ein Container für die grafischen Elemente, sondern koordiniert die Abläufe innerhalb der Anwendung.

4.2 Aufbau der Benutzeroberfläche

Die Benutzeroberfläche ist in mehrere Bereiche aufgeteilt. Der zentrale Bereich ist der Kalender (Planner Workspace im Code), in dem die Termine in unterschiedlichen Kalenderansichten dargestellt werden. Es gibt eine Tagesansicht, eine Wochenansicht und eine Monatsansicht. Die Tagesansicht eignet sich besonders für eine genauere Planung nach Uhrzeit und Raum. Die Wochenansicht gibt einen Überblick über mehrere Tage, während die Monatsansicht eher zur groben zeitlichen Orientierung dient.

Neben den Kalenderansichten gibt es weitere Bereiche, die als sogenannte Docks umgesetzt sind.² Dazu gehören die Terminliste, der Dateneditor, die Datumsnavigation, der globale Filterbereich und die Konfliktanzeige. Durch die verschiebbaren Docks kann jeder Benutzer seinen Arbeitsbereich so einrichten, wie es für ihn am besten passt. Das gewählte Layout lässt sich speichern und später wiederverwenden.

Die grundlegende Aufteilung der Benutzeroberfläche ist in Abbildung 1 dargestellt.

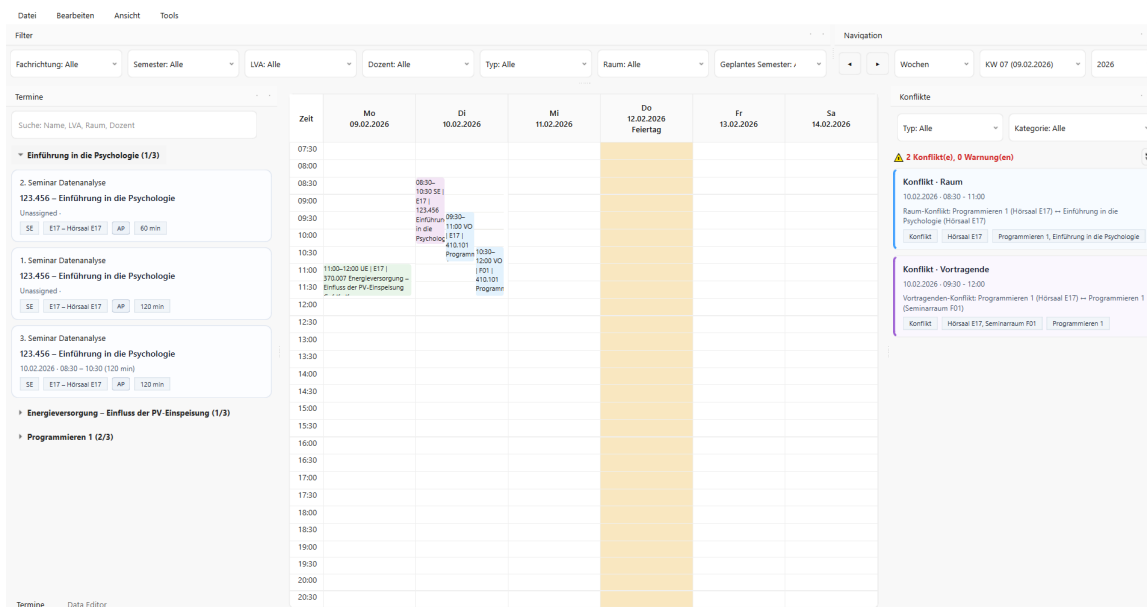


Abbildung 1: Hauptfenster des Planungstools mit Kalenderbereich und Dock-Bereichen

Dadurch wird sichtbar, dass die Kalenderansicht den zentralen Arbeitsbereich bildet, während andere Bereiche wie Terminliste, Filter und Konfliktanzeige als Docks am Rand angeordnet sind.

Die Terminliste dient dazu, vorhandene Termine übersichtlich anzuzeigen. Sie ist besonders wichtig für Termine, die noch nicht vollständig eingeplant wurden. Der Dateneditor wird verwendet, um Termine und Stammdaten wie Lehrveranstaltungen, Räume, Studienrichtungen und freie Zeiträume zu erstellen, zu bearbeiten oder zu löschen. Studiensemester werden dabei als feste globale Auswahlwerte verwendet und reguläre Semester werden automatisch aus Sommer- bzw. Wintersemester und Jahr erzeugt.

²<https://doc.qt.io/qt-6/qdockwidget.html>

Der Filterbereich grenzt die Terminliste gezielt ein, zum Beispiel nach Lehrveranstaltung, Raum, Dozent oder Semester. Die Konfliktanzeige zeigt eine interaktive Liste aller aktuellen Planungskonflikte. Mit dem Navigationsbereich kann der Benutzer beliebig zwischen den Ansichten wechseln und innerhalb der jeweiligen Zeitintervalle navigieren.

4.3 Datenmodell und Datenbeziehungen

Das Datenmodell bildet die Grundlage des Planungstools. Im Mittelpunkt stehen die Termine, da sie die eigentlichen Planungseinträge darstellen. Ein Termin besteht aber nicht nur aus einem Datum und einer Uhrzeit, sondern ist mit weiteren Informationen verbunden. Dazu gehören zum Beispiel die zugehörige LVA, ein Raum, ein Semester, ein Veranstaltungstyp oder eine Gruppe.

Neben den Terminen gibt es weitere Datenbereiche. Dazu zählen Lehrveranstaltungen, Räume, Studienrichtungen und freie Zeiträume. Lehrveranstaltungen können einer Studienrichtung und mehreren Studiensemestern zugeordnet werden. Termine speichern zusätzlich eine Semester-ID, zum Beispiel **SS26** oder **WS26**. Diese Daten werden getrennt gespeichert, damit sie nicht bei jedem Termin erneut eingetragen werden müssen. Stammdaten können dadurch einmal angelegt und anschließend mehrfach verwendet werden.

Die Verbindung zwischen den Daten erfolgt über IDs. Ein Termin speichert also nicht alle Informationen direkt ab, sondern verweist auf den passenden Eintrag. Dadurch können Änderungen an Stammdaten zentral vorgenommen werden.

Neben den Planungsdaten speichert die Anwendung auch Einstellungen und Konfliktpräferenzen in JSON-Dateien. So bleiben persönliche Anpassungen, etwa gespeicherte Layouts oder ausgewählte Konfliktregeln, auch beim nächsten Start erhalten.

4.4 Planungsworkflow

Der Arbeitsablauf im Planungstool beginnt in der Regel mit dem Laden der vorhandenen Daten. Danach können bei Bedarf Stammdaten ergänzt oder angepasst werden. Auf dieser Grundlage werden die einzelnen Termine erstellt und weiter geplant.

Ein Termin kann entweder direkt mit Datum, Uhrzeit, Raum und Semester-ID angelegt werden oder zunächst ohne vollständige zeitliche Zuordnung bestehen. Zusätzlich können Termine als Serientermine mit Enddatum, Periodizität, Ausfallterminen und einzelnen Serien-Ausnahmen gespeichert werden. Dafür verwendet das Tool Startzeit und Dauer statt Start- und Endzeit. So können unzugewiesene Termine ohne Startzeit gespeichert werden und bei Drag-and-Drop ist die Dauer in der Vorschau direkt sichtbar. Noch nicht zugewiesene Termine können später aus der Terminliste in die Kalenderansicht per Drag-and-Drop übernommen werden. Während des Drag-and-Drops werden mögliche Konflikte in der Tages- und Wochenansicht direkt als rote Vorschau angezeigt.

Während der Planung können Filter genutzt werden, um nur bestimmte Termine anzuzeigen. Die Konfliktanzeige hilft zusätzlich dabei, mögliche Probleme schnell zu erkennen.

Der Workflow ist so aufgebaut, dass der Nutzer die Planung weiterhin selbst steuert. Das Tool zeigt Termine und Konflikte übersichtlich an und erleichtert Änderungen.

4.5 Aktualisierung und Zusammenspiel der Komponenten

Da das Planungstool aus mehreren Bereichen besteht, müssen diese nach Änderungen miteinander abgestimmt werden. Wenn ein Termin erstellt, bearbeitet oder verschoben wird, soll diese Änderung nicht nur an einer Stelle sichtbar sein. Auch die Kalenderansicht, die Terminliste und die Konfliktanzeige müssen den neuen Stand übernehmen.

Dafür ist das Hauptfenster als zentrale Stelle der Anwendung wichtig. Nach relevanten Aktionen wie dem Erstellen, Bearbeiten, Löschen oder Verschieben eines Termins wird nicht nur die unmittelbar betroffene Ansicht aktualisiert. Stattdessen stößt das Hauptfenster eine übergreifende Aktualisierung an. Dabei wird zunächst der Kalender mit den aktuellen Planungsdaten neu aufgebaut. Anschließend werden die Dock-Bereiche, wie Terminliste, Dateneditor, Filterbereich und Konfliktanzeige, aktualisiert. Die Konflikte werden ebenfalls neu berechnet.

5 Implementierung

5.1 Entwicklungsumgebung und Projektstruktur

Die Anwendung wurde mit Python in der Version 3.13.2 entwickelt [6]. Für die grafische Benutzeroberfläche wird PySide6 verwendet. Die Implementierung des Planungstools erfolgte in Visual Studio Code.

Der Quellcode ist in mehrere Bereiche aufgeteilt. Im `src`-Verzeichnis befinden sich die wichtigsten Bestandteile des Programms. Dazu gehören die Benutzeroberfläche, verschiedene Services, Dialoge, Hilfsfunktionen und die zentrale Programmlogik. Gestartet wird die Anwendung über die Datei `main.py`.

Um Wartbarkeit und Übersichtlichkeit zu verbessern, wurden alle grafischen Anpassungen in einer separaten `qss`-Datei gebündelt.

Die Planungsdaten liegen in JSON-Dateien im ausgewählten Projektordner. Dadurch kann das Tool mit unterschiedlichen Planungsständen oder Testprojekten verwendet werden.

Für die benötigten Pakete wird eine `requirements.txt`-Datei verwendet. Zusätzlich enthält das Projekt eine `pyproject.toml`, in der weitere Projektinformationen und Einstellungen festgelegt werden.

5.2 Hauptfenster und zentrale Steuerung

Das Hauptfenster der Anwendung wird durch die Klasse `MainWindow` umgesetzt. In diesem Fenster kommen die wichtigsten Teile des Planungstools zusammen. Dazu gehören der Kalenderbereich, die Terminliste, der Dateneditor, die Filter, die Konfliktanzeige und die Datumsnavigation.

Das folgende Beispiel zeigt vereinfacht, wie das Hauptfenster aufgebaut ist. Der Kalenderbereich wird als zentraler Arbeitsbereich gesetzt, während zusätzliche Funktionen wie die Terminliste über Dock-Bereiche eingebunden werden.

```
self.planner = PlannerWorkspace(self, self.ds, ...)
self.setCentralWidget(self.planner)

self.termine_dock = TermineDock(self)
self.addDockWidget(Qt.LeftDockWidgetArea, self.termine_dock)
```

Dadurch bleibt die Kalenderansicht im Mittelpunkt der Anwendung, während ergänzende Bereiche flexibel am Rand des Fensters angeordnet werden können. Das passt gut zum Planungstool, weil die eigentliche Planung sichtbar bleibt und zusätzliche Informationen wie Termine, Filter oder Konflikte trotzdem direkt verfügbar sind.

Als Grundlage wird `QMainWindow` von Qt verwendet.³ Der zentrale Bereich enthält den Kalender mit den Kalenderansichten. Die Dock-Bereiche enthalten die Terminliste, den Dateneditor, die Filter, die Datumsnavigation und die Konfliktanzeige.

Im Hauptfenster werden auch die Menüs und andere Aktionen, wie Aktualisieren, Einstellungen, Undo, Importieren und Exportieren angelegt. Diese Aktionen werden mit den jeweiligen Funktionen der Anwendung verbunden. Dadurch können wichtige Arbeitsschritte direkt über die Menüleiste oder über Tastenkombinationen ausgeführt werden.

Die Verbindung zwischen den einzelnen Bereichen erfolgt über das Signal-System von Qt.⁴ Dadurch können Aktionen aus einem Bereich an andere Bereiche der Anwendung weitergegeben werden. Das folgende Beispiel zeigt eine solche Signal-Verbindung. Wird in der Konfliktanzeige ein Konflikt ausgewählt, wird das entsprechende Signal an den Kalender weitergegeben.

```
self.conflicts_dock.conflict_items_highlight.connect(self.planner.highlight_termine)
```

Dadurch muss die Konfliktanzeige nicht selbst wissen, wie Termine im Kalender dargestellt werden. Sie löst nur ein Signal aus, während der Kalender die Hervorhebung übernimmt.

Eine weitere wichtige Aufgabe des Hauptfensters ist die Aktualisierung der Benutzeroberfläche. Wenn ein Termin erstellt, bearbeitet, gelöscht oder verschoben wird, müssen mehrere Bereiche den neuen Stand anzeigen. Dafür werden zentrale Aktualisierungsmethoden verwendet wie:

```
def refresh_everything(self) -> None:
    self.planner.refresh(emit=False)
    self.refresh_docks()
```

Dabei wird zuerst der Kalender aktualisiert. Anschließend werden über `refresh_docks()` die weiteren Bereiche wie Terminliste, Filter, Dateneditor und Konfliktanzeige neu geladen. Das `emit=False` verhindert, dass der Kalender während der Aktualisierung Signale auslöst, was zu einer Kaskade von Aktualisierungen führen könnte.

Auch Funktionen, die nicht direkt zur Terminplanung gehören, werden im Hauptfenster eingebunden. Dazu zählen zum Beispiel die Layoutverwaltung und die Tastenkombinationen. Der `LayoutManager` speichert und lädt verschiedene Anordnungen der Oberfläche. Die Tastenkombinationen werden in einer eigenen Datei gesammelt und beim Start des Hauptfensters registriert:

```
self.layout_mgr = LayoutManager(self)
install_main_window_shortcuts(self)
```

³<https://doc.qt.io/qt-6/qmainwindow.html>

⁴<https://doc.qt.io/qt-6/signalsandslots.html>

Häufige Aktionen wie Aktualisieren, Ansichtswechsel oder das Erstellen neuer Termine lassen sich dadurch schneller ausführen. Außerdem können eigene Layouts der Benutzeroberfläche gespeichert und verwaltet werden.

5.3 Kalenderansichten

Die Kalenderansichten sind ein zentraler Teil des Planungstools. Sie werden im **PlannerWorkspace** zusammengeführt und ermöglichen es, Termine in unterschiedlichen zeitlichen Ansichten darzustellen. Dabei gibt es eine Tagesansicht, eine Wochenansicht und eine Monatsansicht. Jede Ansicht hat dabei einen etwas anderen Zweck.

Die Tagesansicht ist für die genauere Planung einzelner Termine gedacht. Sie zeigt einen einzelnen Tag mit Uhrzeiten und Räumen. Die Wochenansicht zeigt mehrere Tage nebeneinander. Dadurch bekommt man einen besseren Überblick darüber, wie die Termine innerhalb einer Woche verteilt sind. Die Monatsansicht ist weniger detailliert, zeigt dafür aber einen größeren Zeitraum.

Ein Beispiel für die genaue zeitliche Darstellung einzelner Termine zeigt Abbildung 2.

Zeit	Hörsaal E17	Seminarraum F01	Computerraum LAB3
07:30			
08:00			
08:30	08:30–10:30 SE E17 123-456 Einführung in die Psychologie AP		
09:00			
09:30		09:30–11:00 VO E17 410-101 Programmieren 1 Gr:Gruppe A AP	
10:00			
10:30		10:30–12:00 VO F01 410-101 Programmieren 1 Gr:Gruppe AP	
11:00			
11:30			
12:00			
12:30			
13:00			
13:30			
14:00			
14:30			
15:00			
15:30			
16:00			
16:30			
17:00			
17:30			
18:00			
18:30			

Abbildung 2: Tagesansicht mit zeitlicher Einordnung der Termine

Technisch werden die verschiedenen Ansichten im **PlannerWorkspace** in einem **QStackedWidget** verwaltet.⁵ Dadurch kann immer eine der Ansichten angezeigt werden, während die anderen im Hintergrund vorhanden bleiben. Vereinfacht sieht dieser Aufbau folgendermaßen aus:

```
self.stack = QStackedWidget()
```

⁵<https://doc.qt.io/qt-6/qstackedwidget.html>

```

self.day_table = TimeGridDropTable()
self.week_table = TimeGridDropTable()
self.month_table = MonthDropTable()

self.stack.addWidget(self.day_table)
self.stack.addWidget(self.week_table)
self.stack.addWidget(self.month_container)

```

Die Monatsansicht wird in einem eigenen Container eingebunden, da dort zusätzlich zur Tabelle ein Text mit dem aktuellen Monatsnamen angezeigt wird. Die Tages- und Wochenansicht verwenden jeweils eine zeitbasierte Tabelle mit Räumen oder Wochentagen als Spalten. Die Monatsansicht ist anders aufgebaut, weil sie keinen genauen Zeitraster wie die Tages- oder Wochenansicht verwendet, sondern ein kalendarisches Raster.

Die Termine werden in den Kalenderansichten als eigene Termin-Karten angezeigt. Dadurch können sie besser erkannt, ausgewählt und weiterverwendet werden. Je nach Terminart können die Karten unterschiedlich dargestellt werden. Der Aufbau einer Karte erfolgt dabei über die eigene `TerminCard`-Klasse:

```

class TerminCard(QFrame):
    def __init__(self, termin_id: str, title: str, date: str, time: str,
                  typ: str, raum: str, ap: bool, duration: int = 0, name: str = None, p):
        super().__init__(parent)
        self.setObjectName("TerminCard")

```

Hier wird sichtbar, dass die Karte als eigenes Qt-Widget umgesetzt ist, das von `QFrame` erbt. `QFrame` ist ein einfaches Container-Widget von Qt, das hier als sichtbarer Rahmen für die Termin-Karte dient. Zusätzlich können Termine in der Tages- und Wochenansicht per Drag-and-Drop verschoben werden und dabei während des Verschiebens geprüft werden, ob ein Konflikt entstehen würde.

Zusätzlich kann ein als vorlesungsfrei oder als Feiertag eingetragener Tag in der Kalenderansicht visuell markiert werden. In der Ansicht wird dafür je nach Typ eine Hintergrundfarbe gesetzt:

```

if free_day_type == "feiertag":
    bg = self._free_day_styles.get("holiday_bg")
else:
    bg = self._free_day_styles.get("lecture_bg")

```

Dadurch erkennt der Nutzer sofort, dass an diesem Datum keine reguläre Planung erfolgen soll.

Insgesamt dienen die Kalenderansichten dazu, die Planung nicht nur als Liste von Daten darzustellen, sondern in einen zeitlichen und räumlichen Zusammenhang zu bringen. Der Nutzer kann dadurch besser erkennen, wie Termine verteilt sind und wo Änderungen notwendig sein könnten.

5.4 Termin- und Stammdatenverwaltung

Für die Verwaltung der Termine und Stammdaten gibt es in der Anwendung einen eigenen Dateneditor. Dort können die wichtigsten Daten angelegt, bearbeitet und gelöscht werden. Dazu gehören einzelne Termine sowie Lehrveranstaltungen, Räume, Studienrichtungen und freie Zeiträume. Studiensemester werden im Editor nur als Zuordnung bei Lehrveranstaltungen verwendet, nicht als projektbezogene Stammdaten gepflegt.

Der Dateneditor ist in Abbildung 3 dargestellt.

The screenshot shows a web application window titled "Data Editor". At the top, there are five tabs: "Termine" (selected), "LVAs", "Räume", "Semester", and "Freie Tage". Below the tabs are three buttons: "Hinzufügen", "Bearbeiten", and "Löschen". The main area contains a table with the following data:

	Name	Datum	Von	
1	3. Vorlesung Programmieren			
2	3. Übung Energieversorgung	12.02.2026	12:00	13
3	3. Seminar Datenanalyse	10.02.2026	08:30	10
4	2. Vorlesung Programmieren	10.02.2026	10:30	12
5	2. Übung Energieversorgung			
6	2. Seminar Datenanalyse			
7	1. Vorlesung Programmieren	10.02.2026	09:30	11
8	1. Übung Energieversorgung			
9	1. Seminar Datenanalyse			

At the bottom of the window, there are three tabs: "Termine", "Konflikte", and "Data Editor" (selected).

Abbildung 3: Dateneditor zur Verwaltung von Terminen und Stammdaten

Diese Trennung ist sinnvoll, weil viele Daten mehrfach gebraucht werden. Dadurch

bleiben die Einträge einheitlicher und Änderungen können leichter durchgeführt werden.

Die Bearbeitung der Daten folgt dem CRUD-Prinzip(Erstellen, Anzeigen, Bearbeiten und Löschen). Im Projekt werden diese Aktionen nicht direkt an jeder Stelle neu umgesetzt, sondern über gemeinsame Funktionen im `CrudHandler` behandelt.

```
def read_studiensemester_models(self) -> List[Studiensemester]:
    models: List[Studiensemester] = []
    for item in self.ds.load_studiensemester():
        try:
            models.append(Studiensemester(**item))
        except Exception:
            continue
    return models
```

Diese Methode gehört zum **Read**-Teil des CRUD-Prinzips. Dabei werden gespeicherte Daten geladen, in passende Modellobjekte umgewandelt und zurückgegeben. Fehlerhafte Einträge werden übersprungen.

Die Termin- und Stammdatenverwaltung sorgt somit dafür, dass die Daten geordnet gepflegt und im restlichen Tool verwendet werden können.

5.5 Drag-and-Drop-Funktionalität

Die Drag-and-Drop Funktionalität wurde umgesetzt, damit Termine direkt über die Benutzeroberfläche intuitiv geplant werden können. Der Benutzer muss dadurch Datum, Uhrzeit oder Raum nicht manuell im Dateneditor auswählen, sondern kann einen Termin direkt in den Kalender ziehen oder innerhalb der Kalenderansicht verschieben.

Grundsätzlich gibt es dabei zwei wichtige Funktionalitäten. Einerseits können Termine aus der Terminliste in den Kalender gezogen und einem Datum, einer Uhrzeit oder einem Raum zugewiesen werden und bereits eingeplante Termine können innerhalb des Kalenders verschoben werden. Andererseits ist es auch möglich, einen Termin wieder in die Terminliste zu ziehen und in einen nicht zugewiesenen Zustand zu bringen.

Abbildung 4 zeigt die Terminliste.

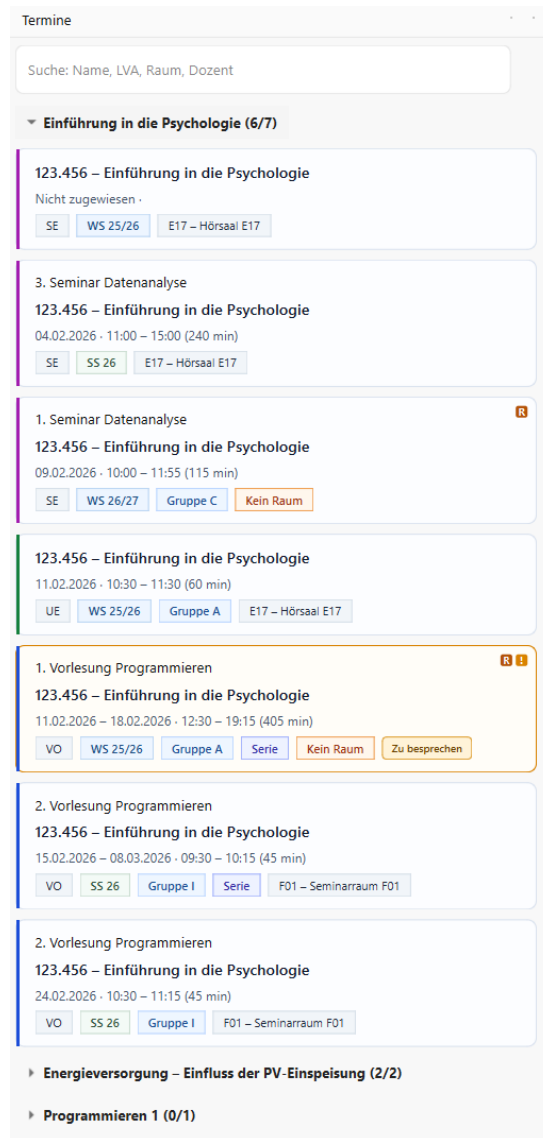


Abbildung 4: Terminliste mit noch nicht vollständig eingeplanten Terminen

Für diese Funktion wird das Drag-and-Drop System von Qt verwendet.⁶ Die Terminliste dient dabei als Ausgangspunkt, aus dem Termine herausgezogen werden können. Die Kalenderansichten übernehmen die Rolle der Zielbereiche, in denen ein Termin abgelegt und dadurch neu zugewiesen werden kann.

Beim Drag-and-Drop wird nur die ID des jeweiligen Termins weitergegeben. Nach dem Ablegen kann die Anwendung anhand dieser ID den passenden Termin laden und die neuen Daten speichern.

```
md = QMimeData()
md.setText(termin_id)
drag = QDrag(self)
drag.setMimeData(md)
```

⁶<https://doc.qt.io/qt-6/dnd.html>

QMimeData wird hier verwendet, um die Informationen für den Drag-and-Drop-Vorgang mitzugeben. In dieser Anwendung reicht es aus zu wissen, welcher Termin verschoben werden soll. Deshalb wird nur die Termin-ID als Text gespeichert und übertragen. QDrag startet anschließend den Drag-Vorgang des Termins. Beim Ablegen werden die gespeicherten Daten an das Ziel-Element übergeben. Die Anwendung kann dann über die ID den richtigen Termin wiederfinden, laden und die geänderten Daten speichern.

Beim Ablegen wird die ID wieder ausgelesen:

```
def dropEvent(self, e: QDropEvent):
    md = e.mimeData()
    termin_id = md.text().strip()
    ... weitere Verarbeitung ...
```

Ein weiterer Teil der Drag-and-Drop Funktion ist die Vorschau während des Verschiebens. In der Tages- und Wochenansicht kann schon während des Ziehens geprüft werden, ob der Termin einen Konflikt auslösen würde. Dafür wird die neue Position nur vorübergehend simuliert.

```
if rows >= 0 and columns >= 0 and self._conflict_checker:
    self._hover_has_conflict = bool(
        self._conflict_checker(self._hover_termin_id, r, c)
    )
```

Während der Termin gezogen wird, prüft die Anwendung ständig, über welcher Kalenderzelle sich der Mauszeiger gerade befindet. Danach wird mit `self._conflict_checker()` überprüft, ob der Termin an dieser neuen Position einen Konflikt auslösen würde.

Eine solche Konfliktvorschau während des Verschiebens zeigt Abbildung 5.

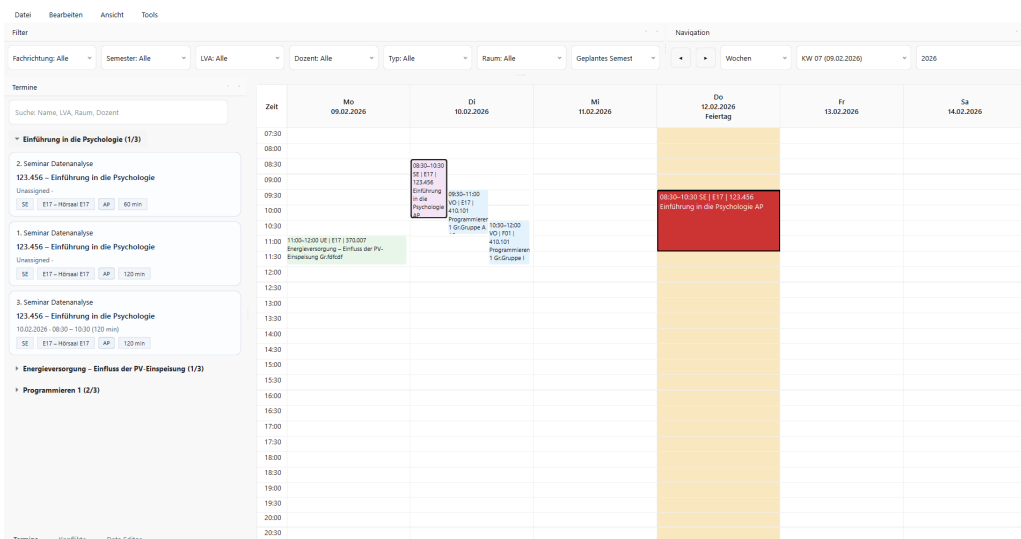


Abbildung 5: Drag-and-Drop-Vorschau eines Termins mit erkanntem Konflikt

Diese Prüfung ist nur eine Vorschau. Die eigentlichen Termindaten werden erst geändert, wenn der Termin wirklich abgelegt wird.

Da diese dynamische Konfliktvorschau während des Ziehens wiederholt ausgeführt wird, kann sie bei sehr vielen Terminen oder auf leistungsschwächeren Geräten die Responsivität der Benutzeroberfläche beeinflussen. Deshalb wurde die Vorschau als optionale Komfortfunktion umgesetzt. In den Einstellungen kann die dynamische Konfliktvorschau deaktiviert werden. In diesem Fall wird während des Ziehens keine rote Konfliktvorschau berechnet, das Verschieben von Terminen bleibt jedoch weiterhin möglich.

5.6 Filterfunktion

Die Filter werden im `GlobalFilterDock` ausgewählt. Dieser Zustand wird als `FilterState` gespeichert und enthält alle aktuell gesetzten Filter.

```
class FilterState:
    studienrichtung: Optional[str] = None
    semester: Optional[str] = None
    lva_id: Optional[str] = None
    gebaeude: Optional[str] = None
    raum_id: Optional[str] = None
    typ: Optional[str] = None
    dozent: Optional[str] = None
    studiensemester: Optional[str] = None
    zu_besprechen: bool = False
```

Wenn der Nutzer dort einen Filter ändert, wird diese Änderung an das Hauptfenster weitergegeben.

```
self.global_filter_dock.filtersChanged.connect(self._on_global_filters_changed)
```

Die Filter ändern nicht die gespeicherten Termindaten. Stattdessen wird nur berechnet, welche Termine zu den aktuell gesetzten Filtern passen. Diese gefilterten Termine werden an die Terminliste übergeben, während der Kalender die Ansicht anhand des gespeicherten `FilterState` aktualisiert.

```
def _on_global_filters_changed(self, fs: FilterState) -> None:
    self.filter_state = fs
    self.planner.set_global_filter_state(fs)
    terms = self._compute_filtered_termine(fs)
    self.termine_dock.set_rows(terms, ...)
```

Die Filterfunktion dient somit vor allem zur besseren Übersicht. Sie hilft dabei, die Planung in kleineren Ausschnitten zu betrachten. Ergänzend zu den globalen Filtern besitzt die Terminliste eine Suchfunktion. Damit kann nach Terminname, Termin-ID, LVA, Raum, Lehrperson oder Besprechungshinweis gesucht werden. Wenn ein passender Termin bereits eingeplant ist, kann über das Kontextmenü der Terminliste direkt zu diesem Termin im Kalender gesprungen werden.

5.7 Konflikterkennung

Die Konflikterkennung soll dabei helfen, mögliche Fehler in der Planung früher zu erkennen. Solche Konflikte und Warnungen können entstehen, wenn Räume, Gruppen oder Lehrpersonen zeitlich überschneiden, wenn Termine auf Feiertage oder vorlesungsfreie Zeiträume fallen, wenn Kapazitäten nicht ausreichen oder wenn Termine für dieselbe Studienrichtung und dasselbe Studiensemester parallel liegen. Solche Bedingungen werden in der Literatur zum University Course Timetabling als typische Einschränkungen beschrieben, die berücksichtigt werden müssen [1].

Das folgende Pseudocode-Beispiel zeigt vereinfacht die Erkennung von Konflikten bei Lehrpersonen. Dafür werden Termine zuerst nach Lehrperson und Datum gruppiert. Anschließend wird jeder Termin innerhalb einer Gruppe mit jedem folgenden Termin verglichen, um zeitliche Überschneidungen zu finden.

```
def detect_lecturer_conflicts(self, termine):
    conflicts = []
    groups = self.group_terms_by_lecturer_and_date(termine)

    for terms in groups.values():
        for i, t1 in enumerate(terms):
            for t2 in terms[i + 1:]:
                if self.times_overlap(t1, t2):
                    conflicts.append(self._create_conflict("lecturer", t1, t2))

    return conflicts
```

Die innere Schleife verwendet `terms[i + 1:]`, damit ein Termin nur mit den nachfolgenden Terminen der Liste verglichen wird. Dadurch wird jedes Terminpaar genau einmal geprüft und doppelte oder selbstbezogene Vergleiche werden vermieden.

Die Prüfung ist regelbasiert aufgebaut. Die Regeln werden aus der Datei `konflikte.json` geladen und können einzeln aktiviert, deaktiviert oder angepasst werden. Die eigentliche Prüfung wird über die Klasse `ConflictDetector` durchgeführt.

```
{
```

```

"name": "Raum-Konflikt",
"key": "room_conflict",
"enabled": true,
"type": "conflict",
"description": "Zwei Termine im selben Raum zur selben Zeit.",
"message_template": "{left_lva} ({left_room}) $\rightarrow$ {right_lva} ({right_room})",
},

```

Die erkannten Konflikte werden in der Konfliktanzeige aufgelistet. Die Darstellung erkannter Konflikte ist in Abbildung 6 zu sehen.

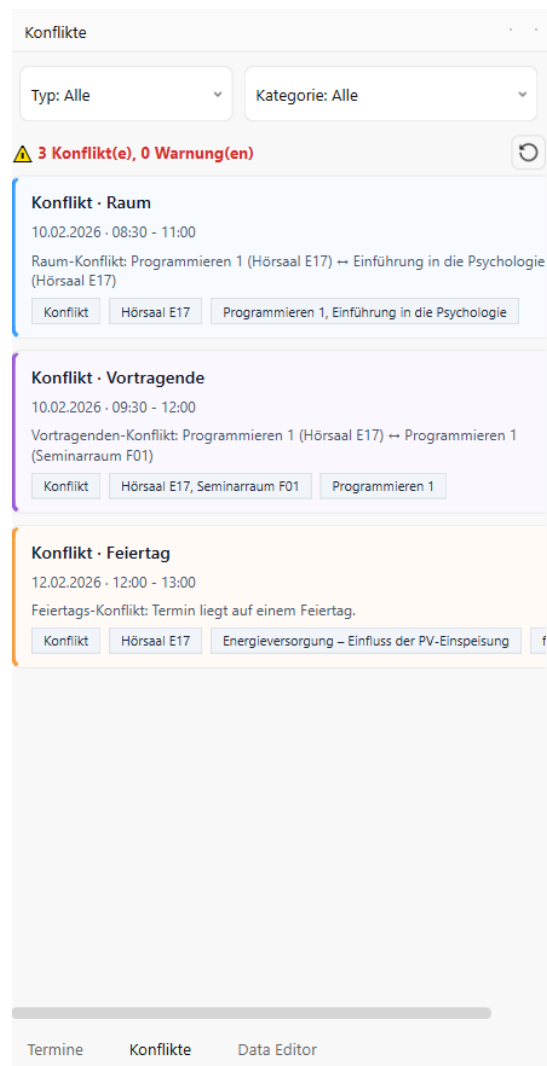


Abbildung 6: Konfliktanzeige mit aufgelisteten Planungskonflikten

Wird ein Konflikt ausgewählt, können die dazugehörigen Termine im Kalender hervorgehoben werden. Dadurch ist sofort erkennbar, an welcher Stelle der Planung das Problem liegt.

Dafür wird die Konfliktanzeige im Hauptfenster über ein Signal mit der Kalenderansicht verbunden.

```
self.conflicts_dock.conflict_items_highlight.connect(self.planner.highlight_termine)
```

Beim Anklicken einer Konfliktkarte werden die betroffenen Termin-IDs gesendet. Diese IDs dienen als eindeutige Referenz auf die Termine, die im Kalender markiert werden sollen.

```
def _on_card_clicked(self, termin_ids):
    if termin_ids:
        self.conflict_items_highlight.emit(termin_ids)
```

In der Kalenderansicht wird zuerst zum ersten betroffenen Termin gesprungen. Danach werden alte Markierungen entfernt, damit nur der aktuell ausgewählte Konflikt sichtbar bleibt. Anschließend werden die betroffenen Termine in der Tages-, Wochen- und Monatsansicht markiert.

```
def highlight_termine(self, ids):
    self._jump_to_first_termin(ids)
    self.clear_conflict_highlights()

    self._highlight_week_cards(ids)
    self._highlight_day_cells(ids)
    self._highlight_month_cells(ids)
```

Die Konflikterkennung wird auch beim Drag-and-Drop verwendet, wobei ein Konflikt als Vorschau angezeigt werden kann. Dafür wird derselbe **ConflictDetector** benutzt wie in der Konfliktanzeige. Die Drag-and-Drop Konfliktvorschau ist aus Performancegründen als optionale Komfortfunktion umgesetzt.

5.8 Import, Export und Speicherung

Die Daten des Planungstools werden in mehreren JSON-Dateien gespeichert. Für die verschiedenen Datenbereiche gibt es eigene Dateien. Beim Start der Anwendung werden diese Dateien geladen und Änderungen werden wieder in diese gespeichert.

Für das Laden und Speichern wird **DataService** verwendet. Dadurch sind die Datei-zugriffe an einer zentralen Stelle gesammelt. Das macht die Speicherung übersichtlicher und erleichtert es, später Änderungen an der Datenverwaltung vorzunehmen, zum Beispiel das Hinzufügen einer Datenbank oder eines zentralen Servers.

Die Anwendung überschreibt eine Datei nicht direkt, sondern schreibt die neuen Daten zuerst in eine temporäre Datei. Erst danach wird die alte Datei ersetzt. Das Ersetzen

erfolgt dabei als atomare Operation auf Betriebssystemebene, sodass entweder die alte oder die neue Datei vollständig vorhanden ist. Dadurch ist die Wahrscheinlichkeit geringer, dass eine JSON-Datei beschädigt wird, falls während des Speicherns ein Fehler auftritt.

```
tmp.write_text(json_text, encoding="utf-8")
tmp.replace(path)
```

Zusätzlich zur normalen Speicherung gibt es eine Import- und Exportfunktion. Beim Projektexport können die aktuellen Projektdaten als JSON oder als Excel-Datei gespeichert werden. Die Excel-Datei enthält getrennte Tabellenblätter für Räume, Lehrveranstaltungen, Termine, Studienrichtungen und freie Zeiträume. So kann ein Planungsstand gesichert oder an einer anderen Stelle wieder verwendet werden. Neben dem vollständigen Projektexport gibt es einen speziellen Export für Lehrende. Dabei können LVAs mit den zugehörigen Lehrpersonen, ein oder mehrere Semester sowie ein Zeitraum ausgewählt werden. Die Termine werden anschließend entweder als Tabellenliste oder als Wochenkalender in eine Excel-Datei geschrieben.

```
files = [
    "raeume.json",
    "lehrveranstaltungen.json",
    "termine.json",
    "studienrichtungen.json",
    "freie_tage.json",
]
```

Beim Import können JSON-, Excel- oder CSV-Dateien eingelesen werden. Die Anwendung erkennt Projektdateien, Raumlisten und LVA-Listen, zeigt neue, geänderte und bereits vorhandene Einträge an und überspringt Einträge mit fehlenden Pflichtverweisen. Danach kann entschieden werden, ob die importierten Änderungen übernommen werden sollen. Dadurch werden vorhandene Daten nicht ungeprüft ersetzt.

Zusätzlich enthält die Anwendung optionale Standarddaten. Diese basieren auf TISS-Daten der TU Wien für Räume und auf ETIT-LVA-Daten mit Stand Juni 2026. Beim Anlegen eines neuen Projekts können diese Daten übernommen, gezielt ausgewählt oder übersprungen werden. Sie dienen als Einstiegshilfe und ersetzen keine direkte Anbindung an TISS.

Außerdem kann in den Einstellungen festgelegt werden, welcher Projektordner verwendet werden soll. Dadurch kann das Tool mit unterschiedlichen Datensätzen arbeiten, zum Beispiel mit Testdaten oder mit den eigentlichen Planungsdaten.

5.9 Semester-Werkzeuge

Für wiederkehrende Planungsarbeiten wurden Semester-Werkzeuge umgesetzt. Damit können Termine eines Quellsemesters in ein Zielsemester kopiert oder Termine eines ausgewählten Semesters gelöscht werden. Beim Kopieren kann das Datum entweder nach der Position im Semester oder nach dem gleichen Kalendertag übertragen werden. Dadurch können bestehende Planungsstände als Grundlage für ein neues Semester verwendet werden, ohne Lehrveranstaltungen, Räume oder andere Stammdaten neu anzulegen.

5.10 Undo/Redo, Shortcuts und Layout-Verwaltung

Neben den eigentlichen Planungsfunktionen wurden auch einige Bedienhilfen umgesetzt. Dazu gehören Undo und Redo, Tastenkombinationen und die Möglichkeit, die Anordnung der verschiedenen Docks zu speichern. Diese Funktionen sollen die Arbeit mit dem Tool erleichtern.

Die Undo- und Redo-Funktion arbeitet mit gespeicherten Zwischenständen der Daten. Bevor eine Änderung durchgeführt wird, wird der aktuelle Zustand als Kopie aller Daten gespeichert. Wenn der Benutzer die Änderung rückgängig macht, wird der frühere Zustand wieder geladen und die Benutzeroberfläche aktualisiert.

```
snapshot = self.undo_service.undo(self.ds)
self.undo_service.restore(self.ds, snapshot)
self.refresh_everything()
```

`undo()` erhält dabei den `DataService` als Parameter, weil es den aktuellen Zustand vor der Wiederherstellung zunächst auf dem Redo-Stack sichert und anschließend den letzten gespeicherten Snapshot zurückgibt. `restore()` schreibt diesen Snapshot zurück in den `DataService`, und `refresh_everything()` aktualisiert anschließend alle sichtbaren Bereiche der Anwendung.

Zusätzlich gibt es Tastenkombinationen für häufig verwendete Aktionen. Dazu gehören unter anderem das Aktualisieren der Ansicht, das Wechseln zwischen Tages-, Wochen- und Monatsansicht, das Erstellen neuer Einträge sowie Import und Export. Einige Shortcuts gelten nur in bestimmten Bereichen der Anwendung, wie das Löschen von Terminen aus dem Kalender, damit sie nicht mit normalen Eingaben in Konflikt kommen.

Das folgende Beispiel zeigt die Zuweisung eines Shortcuts für die Exportfunktion:

```
mw.act_export.setShortcut(QKeySequence("Ctrl+E"))
```

Damit kann der Export direkt über die Tastenkombination `Ctrl+E` gestartet werden, ohne die Aktion über das Menü auswählen zu müssen.

Auch die Benutzeroberfläche kann an die eigene Arbeitsweise angepasst werden. Die einzelnen Docks können verschoben und als Layout gespeichert werden. Beim nächsten

Öffnen der Anwendung kann dieses Layout wieder geladen oder bei Bedarf auf das Standardlayout zurückgesetzt werden.

Diese Funktionen ändern nichts an der Planung selbst, machen die Bedienung aber flexibler. Fehler können leichter rückgängig gemacht werden, häufige Aktionen sind schneller erreichbar und die Benutzeroberfläche muss nicht jedes Mal neu angeordnet werden.

6 Test und Evaluation

6.1 Teststrategie

Die Überprüfung des Planungstools wurde durch manuelle Tests durchgeführt. Softwaretests dienen dazu, Fehler zu finden und zu prüfen, ob zentrale Funktionen wie vorgesehen arbeiten [7]. Dafür wurden typische Arbeitsschritte nachgestellt, wie sie auch bei der späteren Nutzung der Anwendung auftreten können. So sollte überprüft werden, ob die wichtigsten Funktionen zuverlässig funktionieren und ob Änderungen in einem Bereich auch an den passenden Stellen der Anwendung übernommen und sichtbar werden.

Im Mittelpunkt standen dabei die zentralen Funktionen des Tools. Dazu zählen vor allem die Terminverwaltung einschließlich Serienterminen, die Kalenderansichten, Drag-and-Drop, Filter und Suche, Konflikterkennung, Projektordner, Standarddatenimport, Import/Export, Semester-Werkzeuge, Undo/Redo und Speicherung.

6.2 Durchführung der Tests

Für die Testdurchführung wurden die wichtigsten Funktionen der Anwendung überprüft.

Testfall	Durchführung	Ergebnis
Termin erstellen	Ein neuer Termin wird über den Termin-Dialog mit gültigen Daten erstellt.	Der Termin wird gespeichert und erscheint in der Terminliste sowie in der passenden Kalenderansicht, falls er mit Datum und Uhrzeit erstellt wurde.
Serientermin erstellen	Ein Termin wird mit Enddatum, Periodizität und Ausfalltermin angelegt.	Die einzelnen Vorkommen werden im Kalender korrekt angezeigt und der Ausfalltermin wird nicht dargestellt.
Termin bearbeiten	Ein bestehender Termin wird geöffnet und Datum, Uhrzeit oder Raum wird geändert.	Die Änderung wird gespeichert und in Kalenderansicht und Terminliste angezeigt.
Termin löschen	Ein bestehender Termin wird gelöscht.	Der Termin wird aus der Terminliste und aus der Kalenderansicht entfernt.
Termin-Zuweisung entfernen	Ein eingeplanter Termin wird aus dem Kalender entfernt (über Drag-and-Drop)	Der Termin ist nicht mehr im Kalender eingeplant, bleibt aber in der Terminliste vorhanden.
LVA erstellen	Eine neue Lehrveranstaltung wird über den LVA-Dialog erstellt.	Die Lehrveranstaltung wird gespeichert und kann im Termin-Dialog ausgewählt werden.
Raum erstellen	Ein neuer Raum wird in den Stammdaten erstellt.	Der Raum wird gespeichert und steht bei der Terminplanung zur Auswahl.

Tabelle 3: Manuelle Tests der Termin- und Stammdatenverwaltung

Testfall	Durchführung	Ergebnis
Tagesansicht anzeigen	Es wird in die Tagesansicht gewechselt.	Termine des ausgewählten Tages werden korrekt angezeigt.
Wochenansicht anzeigen	Es wird in die Wochenansicht gewechselt.	Termine der ausgewählten Woche werden korrekt angezeigt.
Monatsansicht anzeigen	Es wird in die Monatsansicht gewechselt.	Termine des ausgewählten Monats werden übersichtlich dargestellt.
Navigation im Kalender	Über die Navigation wird zwischen Zeiträumen gewechselt.	Die Kalenderansicht zeigt den neu ausgewählten Zeitraum an. Funktioniert in jeder Ansicht.
Freie Tage anzeigen	Ein freier Tag wird angelegt und anschließend im Kalender betrachtet.	Der freie Tag wird in der Kalenderansicht sichtbar dargestellt.
Termin per Drag-and-Drop zuweisen	Ein Termin wird aus der Terminliste in den Kalender gezogen.	Der Termin erhält die entsprechende zeitliche Zuordnung und erscheint im Kalender.
Termin per Drag-and-Drop verschieben	Ein bereits eingeplanter Termin wird im Kalender an eine andere Stelle verschoben.	Datum, Uhrzeit oder Raum werden angepasst und der Termin wird an der neuen Stelle angezeigt.

Tabelle 4: Manuelle Tests der Kalenderansichten und Drag-and-Drop-Funktion

Testfall	Durchführung	Ergebnis
Filterfunktion	Es werden verschiedene Filter ausgewählt, zum Beispiel nach Semester, Studienrichtung, Studiensemester, LVA, Lehrperson, Typ, Gebäude oder Raum.	Die Anzeige wird jeweils auf die passenden Termine eingeschränkt.
Terminsuche	In der Terminliste wird nach einer LVA, einem Raum oder einer Lehrperson gesucht.	Die Terminliste zeigt nur passende Treffer an und ein eingeplanter Treffer kann im Kalender angesprungen werden.
Filter zurücksetzen	Ein gesetzter Filter wird entfernt.	Die zuvor ausgeblendeten Termine werden wieder angezeigt.
Terminliste aktualisieren	Nach einer Änderung an einem Termin wird die Terminliste kontrolliert.	Die Terminliste zeigt den aktualisierten Termin an.
Konfliktanzeige aktualisieren	Nach einer Änderung an einem Termin wird die Konfliktanzeige kontrolliert.	Die Konfliktanzeige wird entsprechend aktualisiert.

Tabelle 5: Manuelle Tests der Filterung und Aktualisierung

Testfall	Durchführung	Ergebnis
Raumkonflikt erzeugen	Zwei Termine werden zur gleichen Zeit im selben Raum eingeplant.	Der Konflikt wird erkannt und in der Konfliktanzeige dargestellt.
Lehrpersonen-Konflikt erzeugen	Zwei Termine mit derselben Lehrperson werden zeitgleich eingeplant.	Der Konflikt wird erkannt und in der Konfliktanzeige dargestellt.
Gruppenkonflikt erzeugen	Zwei Termine für dieselbe Gruppe werden zeitgleich eingeplant.	Der Konflikt wird erkannt und in der Konfliktanzeige dargestellt.
Studienplan-Warnung erzeugen	Zwei Termine derselben Studienrichtung und desselben Studiensemesters werden zeitgleich eingeplant.	Die Überschneidung wird als Warnung in der Konfliktanzeige dargestellt.
Kapazitätswarnung erzeugen	Ein Termin wird in einen Raum gelegt, dessen Kapazität für die Gruppengröße nicht ausreicht.	Die Kapazitätswarnung wird in der Konfliktanzeige angezeigt.
Termin auf freien Tag legen	Ein Termin wird auf einen eingetragenen freien Tag gelegt.	Der Konflikt wird erkannt und in der Konfliktanzeige dargestellt.
Konflikt anklicken	Ein erkannter Konflikt wird in der Konfliktanzeige ausgewählt.	Die betroffenen Termine werden im Kalender hervorgehoben.
Drag-and-Drop mit Konfliktvorschau	Ein Termin wird in der Tages- oder Wochenansicht auf eine problematische Position gezogen.	Die Vorschau zeigt an, dass an dieser Stelle ein Konflikt entstehen kann.
Dynamische Konfliktvorschau deaktivieren	Die Einstellung für die dynamische Konfliktvorschau wird deaktiviert und ein Termin wird per Drag-and-Drop verschoben.	Während des Ziehens wird keine rote Konfliktvorschau angezeigt, das Verschieben bleibt möglich.
Konfliktanzeige nach deaktivierter Vorschau	Nach deaktivierter Vorschau wird ein Termin so verschoben, dass ein Konflikt entsteht.	Der Konflikt wird nach der Änderung weiterhin in der Konfliktanzeige dargestellt.
Konfliktvorschau auf leistungsschwächerem Gerät	Die Anwendung wird auf einem leistungsschwächeren Gerät mit vielen Terminen verwendet. Ein Termin wird mit aktivierter und anschließend mit deaktivierter dynamischer Konfliktvorschau per Drag-and-Drop verschoben.	Bei aktivierter Vorschau kann das Verschieben verzögert reagieren. Nach dem Deaktivieren der dynamischen Vorschau bleibt das Drag-and-Drop flüssiger.

Tabelle 6: Manuelle Tests der Konflikterkennung

Testfall	Durchführung	Ergebnis
Pflichtfeld leer lassen	Im Termin-Dialog wird ein notwendiges Feld nicht ausgefüllt.	Der Termin wird nicht gespeichert und der Nutzer erhält einen Hinweis.
Dauerberechnung prüfen	Im Termin-Dialog werden Startzeit und Dauer eingetragen.	Die Dauer bzw. die zeitliche Einordnung wird korrekt übernommen.
Löschen bestätigen	Ein Termin oder Stammdateneintrag wird gelöscht.	Die Anwendung fragt nach, ob der Eintrag wirklich gelöscht werden soll.

Tabelle 7: Manuelle Fehler- und Validierungstests

Testfall	Durchführung	Ergebnis
Daten speichern und laden	Änderungen an Terminen oder Stammdaten werden gespeichert und die Anwendung wird erneut geöffnet.	Die gespeicherten Daten sind nach dem erneuten Laden weiterhin vorhanden.
Neues Projekt anlegen	Ein leerer Ordner wird ausgewählt und als neues Projekt vorbereitet.	Die benötigten Projektdateien werden angelegt und der Projektordner wird als Datenpfad gespeichert.
Standarddaten importieren	Beim Anlegen eines Projekts werden die mitgelieferten Standarddaten ausgewählt.	Räume und ETIT-LVAs aus den TISS-basierten Standarddaten werden in das Projekt übernommen.
Projekt exportieren	Ein vorhandener Planungsstand wird als Excel- und alternativ als JSON-Datei exportiert.	Die Exportdatei enthält Räume, LVAs, Termine, Studienrichtungen und freie Zeiträume.
Projekt importieren	Eine zuvor exportierte Datei wird wieder importiert.	Die importierten Daten können übernommen werden.
Raum-Liste importieren	Eine Excel-Datei mit Raum-Daten wird importiert.	Die Anwendung erkennt die Einträge, zeigt neue oder geänderte Daten an und übernimmt die ausgewählten Räume.
Export für Lehrende	Für ausgewählte LVAs bzw. Lehrende und einen Zeitraum wird ein Export für Lehrende erstellt.	Eine Excel-Datei als Terminliste oder Wochenkalender wird erzeugt.
Semester-Werkzeuge verwenden	Termine eines Semesters werden in ein anderes Semester kopiert oder aus einem Semester entfernt.	Die betroffenen Termine werden korrekt kopiert bzw. gelöscht, ohne Stammdaten zu entfernen.
Defekte JSON-Datei importieren	Es wird eine Datei mit ungültiger JSON-Syntax importiert.	Der Import wird abgebrochen und eine Fehlermeldung wird angezeigt.
Undo ausführen	Nach einer Änderung wird die Rückgängig-Funktion verwendet.	Der vorherige Zustand wird wiederhergestellt und die Oberfläche aktualisiert.
Redo ausführen	Nach einem Undo wird die Wiederholen-Funktion verwendet.	Die zuvor rückgängig gemachte Änderung wird erneut übernommen.
Layout speichern	Die Anordnung der Dock-Bereiche wird geändert und gespeichert.	Das gespeicherte Layout kann später wieder geladen werden.

Tabelle 8: Manuelle Tests zu Speicherung, Import/Export und Bedienhilfen

6.3 Bewertung der Ergebnisse

Die manuellen Tests zeigen, dass die zentralen Funktionen des Planungstools zuverlässig arbeiten. Termine lassen sich erstellen, bearbeiten, löschen, verschieben und speichern. Auch die Kalenderansichten, Filter, Konfliktanzeige sowie der Import und Export haben in den getesteten Fällen wie erwartet funktioniert.

Außerdem wurde deutlich, dass typische Planungskonflikte erkannt werden. Dazu gehören zum Beispiel doppelte Raumbelegungen, Überschneidungen bei Lehrpersonen oder Termine an freien Tagen.

7 Diskussion

7.1 Zielerreichung

Das Ziel der Arbeit wurde im Wesentlichen erreicht. Es ist ein Python-basiertes Desktop-Tool entstanden, mit dem Lehrveranstaltungen, Termine, Räume und weitere Planungsdaten zentral verwaltet werden können.

Durch Funktionen wie Kalenderansichten, Filter, Drag-and-Drop und Konflikterkennung wird die Planung übersichtlicher und einfacher zu bearbeiten. Das Tool erstellt die Planung zwar nicht automatisch, unterstützt den Nutzer aber bei typischen Aufgaben und hilft dabei, Fehler schneller zu erkennen.

Damit lässt sich die Forschungsfrage beantworten: Ein solches Planungstool kann durch eine strukturierte Verwaltung der Daten, eine übersichtliche Benutzeroberfläche und eine regelbasierte Prüfung von Konflikten umgesetzt werden.

7.2 Grenzen der Software

Die Software unterstützt den Nutzer bei der Planung, erstellt jedoch keinen Stundenplan automatisch. Die endgültigen Entscheidungen müssen weiterhin manuell getroffen werden.

Außerdem ist die Anwendung für die lokale Nutzung ausgelegt. Ein gleichzeitiger Zugriff durch mehrere Nutzer ist in dieser Version nicht vorgesehen.

Die Speicherung in JSON-Dateien reicht für den aktuellen Umfang aus, könnte bei größeren Datenmengen jedoch unübersichtlich oder unpraktisch werden.

7.3 Erweiterungsmöglichkeiten

Mögliche Erweiterungen wären zum Beispiel eine Datenbankanbindung und ein Mehrbenutzerbetrieb. Auch eine halbautomatische Planung wäre eine sinnvolle Ergänzung. Das Tool könnte dann passende Termine, freie Räume oder mögliche Alternativen vorschlagen und den Nutzer bei der Entscheidung unterstützen.

Eine Anbindung an bestehende Universitätssysteme wäre ebenfalls hilfreich, damit Daten nicht doppelt gepflegt werden müssen und die Anwendung besser in bestehende Abläufe integriert werden kann.

8 Fazit und Ausblick

In dieser Arbeit wurde ein Python-basiertes Planungstool für die Lehrveranstaltungsplanung entwickelt. Ziel war es, die Planung übersichtlicher, strukturierter und weniger fehleranfällig zu gestalten.

Das Tool verwaltet Termine, Lehrveranstaltungen, Räume, Studienrichtungen und freie Zeiträume in lokalen Projektordnern und unterstützt zusätzlich Serientermine, Standarddatenimport, Semester-Werkzeuge sowie Excel-Exporte für Lehrende. Funktionen wie Kalenderansichten, Filter, Drag-and-Drop und Konflikterkennung unterstützen den Nutzer bei der Bearbeitung der Planung. Die Daten werden dabei lokal in JSON-Dateien gespeichert. Das Tool übernimmt die Planung nicht automatisch, erleichtert aber typische Arbeitsschritte und macht mögliche Konflikte schneller sichtbar.

Für die Zukunft wären eine Datenbankanbindung, ein Mehrbenutzerbetrieb und die Integration in bereits vorhandene Universitätssysteme sinnvolle Erweiterungen.

Literatur

- [1] Babaei, H., Karimpour, J., & Hadidi, A. (2015). A survey of approaches for university course timetabling problem. *Computers & Industrial Engineering*, 86, 43–59. <https://doi.org/10.1016/j.cie.2014.11.010>
- [2] Bray, T. (2017). *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC 8259. <https://www.rfc-editor.org/rfc/rfc8259>
- [3] Python Software Foundation. (o. J.). *Python Documentation*. <https://docs.python.org/3/>
- [4] Qt Group. (o. J.). *Qt Documentation*. <https://doc.qt.io/qt-6/>
- [5] Qt Group. (o. J.). *Qt for Python Documentation*. <https://doc.qt.io/qtforpython-6/>
- [6] Python Software Foundation. (2025). *Python 3.13.2 Release*. <https://www.python.org/downloads/release/python-3132/>
- [7] Sommerville, I. (2016). *Software Engineering* (10th ed., Global Edition). Pearson.
- [8] National Institute of Standards and Technology. (o. J.). *Usability*. Computer Security Resource Center Glossary. <https://csrc.nist.gov/glossary/term/usability>